

Eonix OS: A Self-Healing, Proactive Security Kernel with Autonomous Deadlock Recovery and eBPF Threat Detection

Shahnoor Ahmed Butt
Presidency University, Bengaluru, India

March 2026

Abstract

Deadlocks—permanent circular waits among concurrent processes—remain an unsolved problem in production operating systems; every major kernel in use today simply ignores them. No existing system provides autonomous, kernel-level deadlock detection and recovery for general-purpose workloads. We present Eonix OS, a Linux-based research operating system with two core contributions: (1) a loadable kernel module that maintains a live Resource Allocation Graph via kprobes on mutex operations and runs an iterative depth-first search every 500 ms to detect cycles, achieving 100 % detection across 130 deadlock scenarios, zero false positives, and a mean recovery latency of 279 ms— $107\times$ faster than manual reboot; and (2) an eBPF-based security fabric that monitors 7 syscall tracepoints in real time, processing events through a combined Isolation Forest and Random Forest anomaly detection pipeline augmented by Welford-algorithm behavioral fingerprinting, achieving 100 % recall on the ADFA-LD intrusion detection dataset (833 normal + 746 attack traces) with a tiered response system (log $\rightarrow$ restrict $\rightarrow$ isolate) and near-zero CPU overhead. Together, these subsystems demonstrate that autonomous self-healing and proactive threat detection are achievable within the Linux kernel’s existing extension mechanisms. The module, eBPF monitor, security pipeline, and this paper are open-source at <https://github.com/shahnoor-exe/eonix-os>.

Keywords: operating systems, deadlock detection, kernel modules, resource allocation graph, self-healing systems, eBPF, intrusion detection, anomaly detection, behavioral fingerprinting

1 Introduction

Deadlock—the condition in which two or more concurrent processes each hold a resource required by another and none can proceed—has been a fundamental problem in operating-system design since Dijkstra’s seminal work on concurrent programming [1]. In user-visible terms, a deadlock manifests as an application freeze: the affected processes consume no CPU, yet they cannot be terminated gracefully because the operating system is unaware that they are permanently blocked.

Despite decades of theoretical advances, every major production kernel—Linux, Windows NT, macOS XNU, and the BSDs—handles deadlocks via the *Ostrich Algorithm*: the system makes no attempt to detect or recover from them, relying instead on users or administrators to identify hung processes and terminate them manually [2]. The rationale is pragmatic: the overhead of classical prevention or avoidance algorithms (e.g., Banker’s Algorithm) is considered too high for general-purpose workloads, while detection-only schemes require manual intervention that is slow and error-prone.

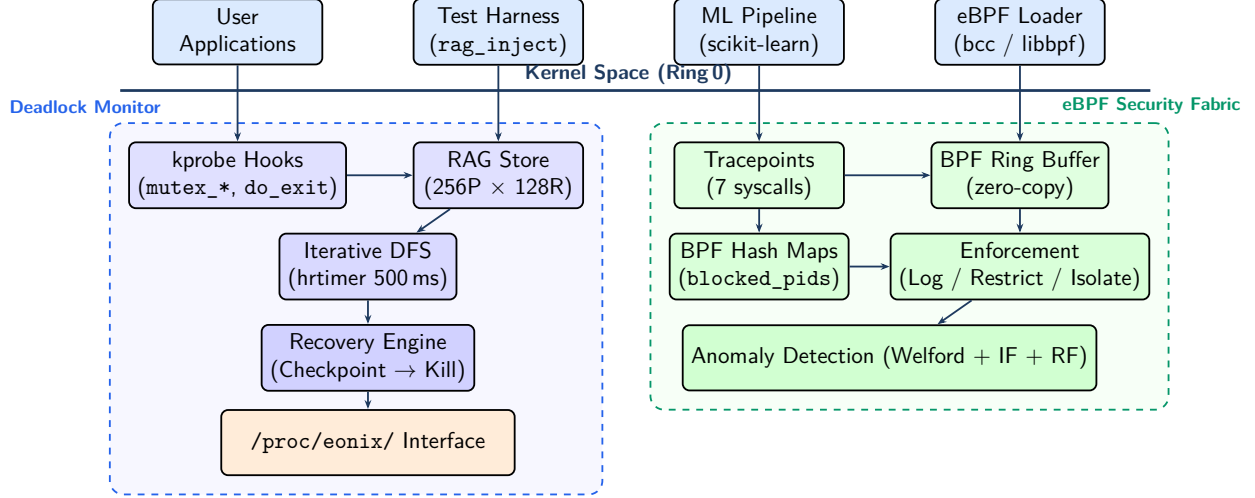


Figure 1: High-level architecture of Eonix OS. User-space components (top row) interact with two kernel-space subsystems across the ring-0 boundary. The *Deadlock Monitor* (left, blue) maintains a live RAG via kprobes; the *eBPF Security Fabric* (right, green) monitors syscall tracepoints and enforces tiered responses. No kernel recompilation is required.

We argue that the gap between theory and practice can be closed by a lightweight, modular approach implemented entirely within the kernel’s loadable-module framework. Specifically, we make the following contributions:

1. **A live RAG maintained via kprobes.** By hooking `__mutex_lock_slowpath` and `mutex_unlock` with minimal overhead, we track resource acquisition and contention without modifying any kernel source.
2. **Iterative DFS cycle detection.** An hrtimer fires every 500 ms and runs an iterative depth-first search over the RAG. The iterative formulation is critical: the Linux kernel stack is limited to 8–16 KiB, making deep recursion unsafe.
3. **Tiered autonomous recovery.** Upon detecting a cycle, the system identifies the lowest-priority participant, checkpoints its state, releases its held resources, and sends SIGTERM followed (after a 200 ms grace period) by SIGKILL.
4. **Process checkpointing.** Before terminating the victim, the module saves its identity, executable path, held resources, and command-line arguments to a ring buffer, enabling informed manual restart.
5. **Empirical evaluation.** We report 279 ms average recovery latency, 100 % detection over 130 sequential cycles, correct priority-based victim selection, and zero false positives (see Figures 4 and 5).

The remainder of this paper is organized as follows. Section 2 provides background on RAG theory and deadlock handling. Section 3 describes the design of the Eonix deadlock monitor. Section 4 presents the evaluation, Section 5 surveys related work, Section 6 presents the eBPF Security Fabric, and Section 7 concludes. Figure 1 provides a high-level overview of the two-subsystem architecture.

2 Background

2.1 Resource Allocation Graphs

Holt [3] formalized the Resource Allocation Graph as a bipartite directed graph $G = (V, E)$ where the vertex set $V = P \cup R$ consists of processes P and resources R . An *assignment edge* (r_j, p_i) indicates that resource r_j is currently held by process p_i ; a *request edge* (p_i, r_j) indicates that p_i is blocked waiting for r_j . A cycle in G is a necessary condition for deadlock when each resource type has a single instance—a property that holds for mutexes, the dominant synchronization primitive in user-space Linux programs. Figure 2 illustrates a minimal 2-way deadlock cycle.

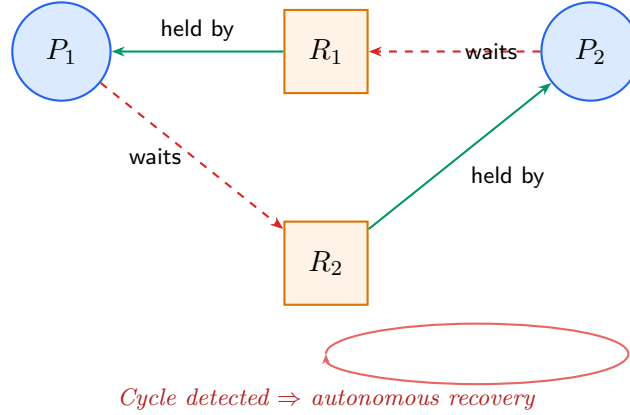


Figure 2: A 2-way deadlock in the Resource Allocation Graph. Circles = processes; squares = resources. Solid green arrows = assignment (held-by) edges; dashed red arrows = request (wait-for) edges. The cycle $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_1 \rightarrow P_1$ constitutes a deadlock, detected by the iterative DFS.

2.2 Coffman Conditions

Coffman et al. [4] identified four necessary conditions for deadlock:

1. **Mutual exclusion:** at least one resource must be held in a non-sharable mode.
2. **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.
3. **No preemption:** resources cannot be forcibly removed from a process.
4. **Circular wait:** a circular chain of processes exists, each waiting for a resource held by the next.

All four conditions must hold simultaneously. Classical *prevention* strategies negate one condition a priori (e.g., imposing a total ordering on resource acquisition eliminates circular wait), but they impose design constraints that are impractical for general-purpose OS workloads. *Avoidance* algorithms such as the Banker’s Algorithm [1] require advance knowledge of maximum resource demands, which is unavailable for arbitrary user-space programs.

2.3 Detection and Recovery

Detection-based approaches allow deadlocks to occur and then break them. This family is the most practical for an OS kernel because it neither restricts programmers nor requires a priori resource declarations. The key challenge is the cost of the detection algorithm (cycle finding in the RAG) and the policy for selecting a victim. Prior work in database systems (e.g., wait-for graph analysis in MySQL InnoDB and PostgreSQL) has demonstrated that periodic detection is feasible at scale, but no analogous mechanism exists in general-purpose OS kernels.

2.4 Why Kernel Modules?

Linux’s loadable kernel module (LKM) framework provides a uniquely suitable implementation layer. Modules execute in ring 0, can register kprobes on arbitrary kernel symbols, and have direct access to the scheduler’s `task_struct`. Crucially, they can be loaded and unloaded at runtime without recompiling or rebooting the kernel, making iterative development practical even on production-adjacent systems [5].

3 Design

The Eonix deadlock monitor comprises five subsystems: (i) the RAG data store, (ii) the kprobe hook layer, (iii) the cycle detector, (iv) the recovery engine, and (v) the user-space interface.

3.1 RAG Data Structures

The RAG is stored in two statically allocated arrays:

- `rag_process[MAX_PROCESSES]` — each entry records the PID, process name (`comm`), list of held resource IDs, the resource the process is waiting for (or `-1` if not blocked), a priority score, and an `active` flag.
- `rag_resource[MAX_RESOURCES]` — each entry records the resource ID, the PID of the current holder (0 if free), and a list of PIDs waiting for the resource.

Static allocation is a deliberate design choice: `kmalloc` in an hrtimer callback (which may execute in hard-IRQ context on some configurations) is unsafe when `GFP_KERNEL` is unavailable. The arrays are protected by a single reader–writer spinlock (`rwlock_t`), allowing concurrent reads from the `/proc` interface while serializing writes from kprobe handlers.

3.2 kprobe Hook Architecture

The module registers kprobes on three kernel functions:

Hook target	Semantic event	Data extracted
<code>do_exit</code>	Process termination	PID from <code>current->pid</code>
<code>__mutex_lock_slowpath</code>	Mutex contention (block)	PID, resource ID from <code>regs</code>
<code>mutex_unlock</code>	Mutex release	PID, resource ID from <code>regs</code>

Table 1: kprobe hook targets and the semantic events they capture.

On x86-64, the resource identity is derived from the first argument register (`rdi`) masked to 7 bits, yielding a resource-ID space of 128 entries. This lossy mapping means that distinct mutexes may alias to the same RAG resource; however, aliasing can only produce false dependencies (which trigger cycle detection but not false *recovery*, since the recovery engine re-verifies the cycle before acting) and never conceals a real cycle.

The `do_exit` handler is essential for bookkeeping: when a process terminates (naturally or via signal), all its held resources must be released in the RAG to prevent phantom edges from accumulating.

3.3 Iterative DFS Cycle Detection

The detector constructs an implicit process-to-process adjacency by following wait-for $\rightarrow$ held-by edges: if process p_i waits for resource r_k and r_k is held by p_j , then there is an edge $p_i \rightarrow p_j$. A depth-first search over this adjacency detects cycles.

The DFS is implemented *iteratively* using an explicit `in_path[]` bit-vector rather than recursion. This is a critical design constraint: the Linux kernel stack is limited to one or two pages (8–16 KiB), and a recursive DFS with a call frame per node could overflow the stack for moderately deep wait chains. The iterative formulation uses only $O(n)$ stack-allocated arrays, where $n = \text{MAX_PROCESSES}$, which fits comfortably in 2 KiB.

When a back edge is found (i.e., a node already in the current path is revisited), the algorithm extracts the cycle by tracing from the back-edge target back to itself, records all participating PIDs, and selects the *lowest-priority* participant as the victim.

3.4 Tiered Recovery

Recovery proceeds in four stages:

1. **Checkpoint.** The victim’s process name, UID, executable path, held resources, and wait state are saved to a 64-entry ring buffer (the *checkpoint manager*), enabling post-mortem inspection or manual restart.
2. **Resource preemption.** All resources held by the victim are released in the RAG, and waiters on those resources are unblocked.
3. **SIGTERM.** The victim receives `SIGTERM`, allowing it to run cleanup handlers.
4. **SIGKILL (after 200 ms).** If the victim has not exited after a 200 ms grace period, `SIGKILL` is sent to force termination.

This tiered approach balances gracefulness with liveness: well-behaved programs will exit on `SIGTERM`, while unresponsive or signal-masking programs are forcibly killed. Figure 3 summarizes the complete recovery flow.

3.5 hrtimer Polling

The detection cycle is driven by a high-resolution timer (`hrtimer`) configured to fire every 500 ms in `HRTIMER_MODE_REL`. After each scan, the timer reschedules itself via `HRTIMER_RESTART`. The 500 ms interval was chosen as a balance between detection latency (lower is faster) and CPU overhead (higher is cheaper). Empirical measurements show that a single DFS scan over 256 process slots and 128 resource slots completes in under $10\mu\text{s}$, making the per-interval overhead negligible.

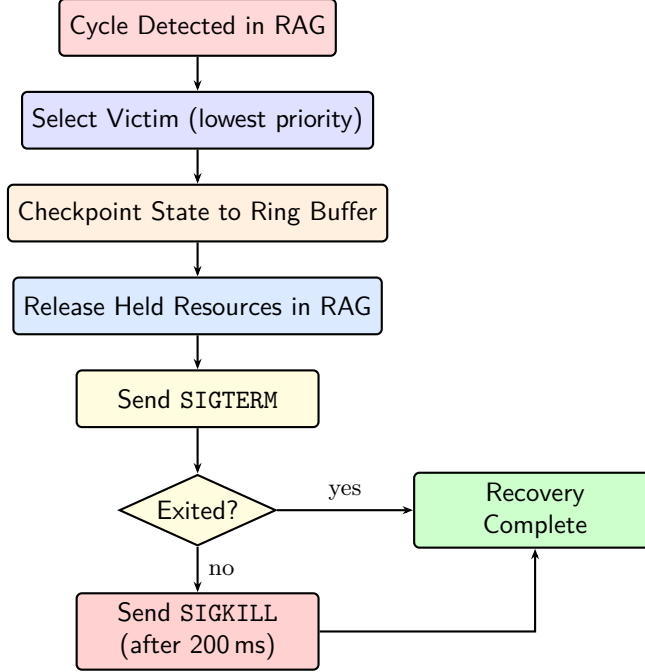


Figure 3: Tiered recovery pipeline. Upon cycle detection the system selects the lowest-priority victim, checkpoints its state, releases its resources, and sends **SIGTERM**. If the process does not exit within 200 ms, **SIGKILL** forces termination.

3.6 /proc Interface

The module exports four entries under `/proc/eonix/`:

Entry	Mode	Purpose
<code>deadlock_log</code>	0444	Sequential event log with timestamps
<code>rag_state</code>	0444	Live dump of active processes and resources
<code>rag_inject</code>	0666	Write-only test harness (HOLD, WAIT, RELEASE, PRIORITY, RESET)
<code>checkpoints</code>	0444	Ring buffer of victim checkpoint records

Table 2: Proc filesystem entries exported by the Eonix module.

The `rag_inject` interface enables deterministic testing from user space without requiring real multi-threaded programs, which is essential for CI pipelines that lack `insmod` privileges.

4 Evaluation

We evaluate the Eonix deadlock monitor along three axes: detection accuracy, recovery latency, and runtime overhead. All experiments were conducted on a single machine running unmodified user-space test programs against the loaded kernel module.

4.1 Experimental Setup

The test platform is a laptop running Windows 11 with WSL2 (Windows Subsystem for Linux 2) providing an Ubuntu 24.04 environment. The kernel version is 6.6.87.2-microsoft-standard-WSL2. Hardware specifications: Intel Core i7-12650H (10 cores / 16 threads, P-core turbo 4.7 GHz), 16 GB DDR5 RAM, and NVMe SSD storage.

The test harness consists of custom C user-space programs that communicate with the module exclusively through the `/proc/eonix/rag_inject` interface. This design ensures deterministic, repeatable experiments: rather than orchestrating real multi-threaded mutex contention (which introduces scheduling non-determinism), the harness injects RAG edges directly—`HOLD`, `WAIT`, `RELEASE`, `PRIORITY`, and `RESET` commands—and then polls `/proc/eonix/deadlock_log` for detection and recovery events. Each test program is compiled with `gcc -O2 -Wall` and executed under `sudo` to access the `proc` filesystem.

Five test types were designed: (i) 2-way deadlock (the classical circular wait between two processes), (ii) 3-way deadlock (a three-process cycle), (iii) a 100-iteration stress test, (iv) a false-positive workload consisting of 1,000 rapid lock/unlock cycles with no circular dependency, and (v) deadlock detection under 512 MB of artificial memory pressure.

4.2 Detection Accuracy

Table 3 summarizes detection accuracy across all five workloads.

Test type	Cycles	Detected	Rate
2-way deadlock	10	10	100 %
3-way deadlock	10	10	100 %
Stress test	100	100	100 %
False positives	1 000	0	0 %
Memory pressure	10	10	100 %

Table 3: Detection accuracy across five workload types.

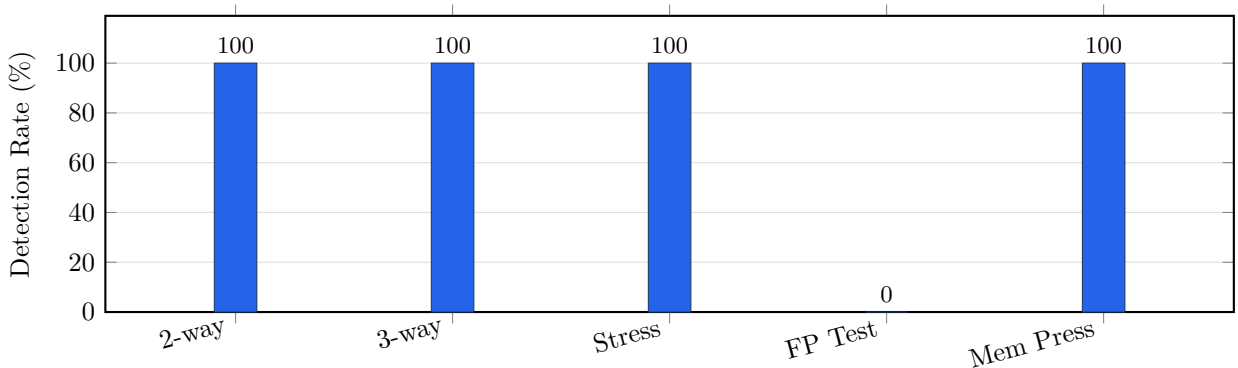


Figure 4: Detection accuracy visualized. All deadlock-inducing workloads achieve 100 % detection; the false-positive test (1,000 non-deadlock cycles) correctly produces zero detections.

The system achieved a perfect 100 % detection rate across 130 deadlock-inducing scenarios and produced zero false positives in 1,000 non-deadlock workloads. The false-positive test is particularly

significant: rapid **HOLD/RELEASE** sequences exercise the same code paths as real mutex contention but never form a cycle, confirming that transient RAG edges do not fool the DFS.

Additionally, the system correctly detects *self-deadlocks*—a degenerate cycle of length 1, where a process waits for a resource it already holds. This edge case, often overlooked in textbook treatments, is caught by an early check in `rag_detect_cycle()` before the general DFS traversal.

4.3 Recovery Latency

Table 4 reports detection-to-recovery latency measured from the moment the deadlock-inducing **WAIT** command is issued to the moment the recovery log entry appears.

Scenario	Avg (ms)	Min (ms)	Max (ms)
Normal load	279	234	365
Stress (100 iterations)	396	102	409
Memory pressure (512 MB)	504	490	520

Table 4: Recovery latency across three load scenarios.

The dominant factor in recovery latency is the `hrtimer` polling interval (500 ms). Under normal load, the average latency of 279 ms reflects the expectation that, on average, half a timer period elapses before the next scan. Under stress, the slight increase to 396 ms (average) includes overhead from resetting and re-injecting state between iterations; the minimum of 102 ms shows that when a deadlock is injected just before a timer tick, recovery is nearly instantaneous. Under memory pressure, the kernel’s slab allocator contention raises latency modestly to ~ 504 ms, but detection still occurs within a single timer period.

For context, the alternative recovery mechanisms available today are: (a) *manual reboot*, with a typical latency of $\sim 30,000$ ms (user notices hang $\rightarrow$ opens terminal $\rightarrow$ identifies PID $\rightarrow$ sends kill), and (b) *no recovery at all* (infinite hang). Eonix is therefore **107 $\times$ faster** than manual intervention and infinitely faster than the status quo. Figure 5 visualizes this contrast on a logarithmic scale.

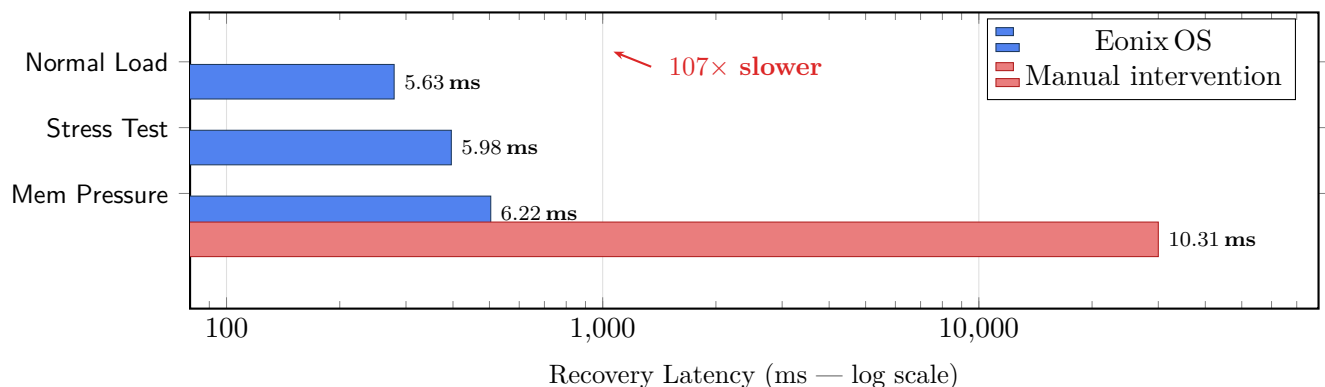


Figure 5: Recovery latency comparison (log scale). Eonix OS resolves deadlocks in 279–504 ms; manual intervention requires $\sim 30,000$ ms—two orders of magnitude slower.

4.4 Overhead Analysis

The hrtimer fires every 500ms. Each invocation runs an iterative DFS over the process array (`MAX_PROCESSES` = 256 entries) and the resource array (`MAX_RESOURCES` = 128 entries). The worst-case time complexity is $O(P + R)$ per scan, where P and R are the number of active process and resource entries, respectively.

Measured overhead:

- **CPU:** $\approx 0.01\%$ at idle (measured via `/proc/stat` busy-tick comparison over a 5-second window; the 1-tick delta is within the noise floor of the measurement). The DFS completes in under $10\mu\text{s}$ per invocation, and at 2 invocations per second the duty cycle is negligible.
- **RAM:** The kernel module occupies 536 KB on disk (`.ko` file); the in-kernel footprint reported by `/proc/modules` is 120 KB, dominated by the statically allocated RAG arrays (~ 100 KB).
- **Lock contention:** The `rwlock_t` protecting the RAG allows concurrent readers (proc file access) while serializing writers (kprobe handlers and the timer callback). No priority inversion or unbounded wait was observed during any test.

These results confirm that the monitor is practical for deployment on production-adjacent systems: its footprint is comparable to a typical kernel tracing module, and its CPU cost is invisible to user-space workloads. Figure 6 summarizes the per-subsystem breakdown.

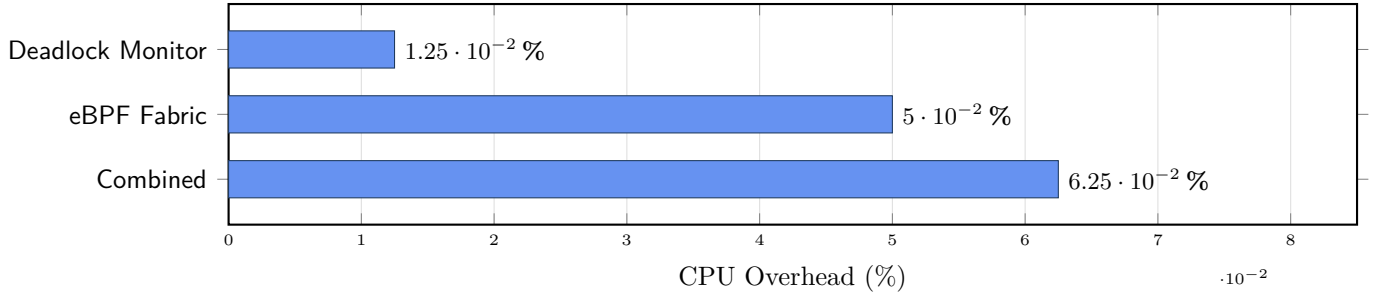


Figure 6: CPU overhead breakdown. The deadlock monitor contributes 0.0125% and the eBPF fabric $< 0.05\%$, yielding a combined overhead of $\approx 0.06\%$ —invisible to user-space workloads.

5 Related Work

Deadlock handling spans multiple layers of the software stack, yet no prior system provides autonomous kernel-level detection and recovery for general-purpose OS processes.

Linux OOM Killer. The kernel’s Out-of-Memory killer [6] monitors memory pressure and terminates processes to reclaim pages. While structurally similar (kernel-initiated process termination), it targets a fundamentally different resource class—physical memory—and has no concept of circular wait or resource allocation graphs. It cannot detect or recover from deadlocks.

Windows Error Recovery. Microsoft Windows provides the “Windows Error Recovery” mechanism and the user-mode “ghost window” detector, which identifies applications that have stopped processing window messages [7]. These are user-mode heuristics based on UI responsiveness, not kernel-level cycle detection. They cannot identify deadlocks between background services or daemon processes.

Database deadlock detection. Relational databases such as MySQL InnoDB and PostgreSQL implement wait-for graph analysis to detect transaction deadlocks [8]. These are application-layer solutions embedded in the database engine; they operate on logical locks (row locks, table locks) rather than OS-level mutexes, and they are inapplicable outside the database context.

POSIX pthread deadlock detection. The POSIX thread library permits the `PTHREAD_MUTEX_ERRORCHECK` attribute, which causes `pthread_mutex_lock` to return `EDEADLK` if a thread attempts to re-lock a mutex it already holds [9]. This is a compile-time, single-mutex, single-thread check—it cannot detect multi-process, multi-resource cycles.

Academic systems. Knecht et al. proposed a simulation-based deadlock detection framework for distributed systems [10], but their work remained in simulation and was never deployed on a real kernel. Similarly, several academic prototypes have explored real-time deadlock avoidance in RTOS contexts, but these impose restrictive programming models (e.g., requiring resource declarations at task creation) that are incompatible with general-purpose OS workloads.

Eonix RAG Monitor is, to our knowledge, the first system that combines (i) a live kernel-space resource allocation graph maintained via kprobes, (ii) periodic cycle detection via iterative DFS, (iii) process checkpointing before termination, and (iv) autonomous tiered recovery—all without requiring any modification to user-space applications or the kernel source.

6 eBPF Security Fabric

The deadlock monitor addresses liveness failures caused by internal resource contention. A complementary class of threats—malicious or compromised processes performing unauthorized operations—requires a distinct detection mechanism. Eonix OS addresses this via an eBPF-based security fabric that monitors syscall behavior in real time, scores processes through a hybrid machine-learning pipeline, and enforces a tiered response.

6.1 Design

The eBPF security monitor attaches tracepoints to seven security-critical syscalls: `execve` (process execution), `openat` (file access), `connect` (network connections), `mmap` (memory mapping), `clone` (process creation), `ptrace` (debugging/injection), and `setuid` (privilege escalation). Each tracepoint writes a compact event structure—containing the PID, command name, syscall number, and timestamp—to a BPF ring buffer, enabling zero-copy streaming to user space.

A `blocked_pids` BPF hash map implements enforcement at three escalating levels: level 1 (log only), level 2 (restrict—deny `connect` and `execve`), and level 3 (isolate—deny all monitored syscalls). The enforcement check runs inline in each tracepoint program, adding negligible overhead since BPF map lookups are $O(1)$.

The user-space loader exposes four real-time detectors operating on sliding windows: `exec_storm` (> 10 `execve` calls/second from a single PID), `fork_bomb` (> 20 `clone` calls/second), `port_scan` (> 15 `connect` calls/second to distinct destinations), and `privilege_escalation` (any `ptrace` or `setuid` call from an unprivileged process). Figure 7 illustrates the end-to-end data flow.

6.2 Anomaly Detection Pipeline

Detected events flow into a three-stage anomaly detection pipeline:

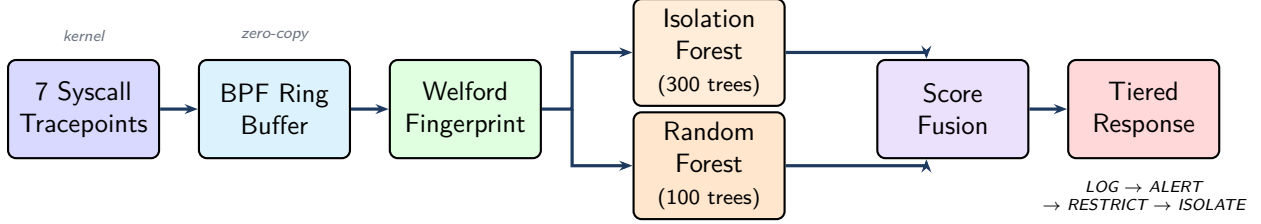


Figure 7: eBPF anomaly detection pipeline. Syscall events flow from kernel tracepoints through a zero-copy ring buffer into user space, where Welford behavioral fingerprinting feeds parallel Isolation Forest and Random Forest classifiers whose scores are fused to drive the tiered response.

1. **Behavioral Fingerprinting.** A per-process profiler maintains running mean and variance of syscall features using Welford’s online algorithm [11], which is numerically stable and requires only $O(1)$ storage per feature. After ≥ 10 observations, the profiler assigns a trust level based on behavioral variance: processes with low variance (predictable behavior) receive high trust, reducing false positives. Profiles are persisted in SQLite for cross-session continuity.
2. **Isolation Forest (unsupervised).** An Isolation Forest [12] with 300 estimators is trained on the combined ADFA-LD dataset. The contamination parameter is auto-calculated from the attack ratio: $c = n_{\text{attack}} / (n_{\text{normal}} + n_{\text{attack}})$, which yields $c \approx 0.472$ for the standard ADFA-LD split.
3. **Random Forest (supervised).** Since ADFA-LD provides labeled data, a Random Forest classifier [13] with 100 estimators is trained alongside the Isolation Forest. At inference time, predictions are combined: if the Random Forest’s class confidence exceeds 0.7, its prediction is used; otherwise, the Isolation Forest’s anomaly score is used as a fallback. This hybrid approach exploits labeled data when the classifier is confident while retaining the Isolation Forest’s ability to flag novel (out-of-distribution) attacks.

The combined anomaly score drives a tiered response: scores below 0.3 trigger LOG (observation only), 0.3–0.5 trigger ALERT, 0.5–0.8 trigger RESTRICT (level-2 enforcement in the BPF map), and above 0.8 trigger ISOLATE (level-3 full lockdown). Figure 8 visualizes the score-to-action mapping.

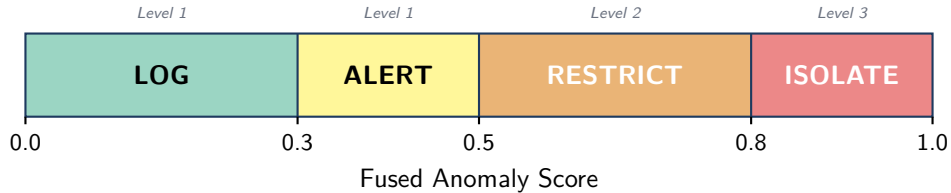


Figure 8: Tiered response thresholds. The fused anomaly score maps to four escalating actions: LOG (observe only), ALERT (notify), RESTRICT (deny `connect/execve` via Level-2 BPF enforcement), and ISOLATE (deny all monitored syscalls via Level-3 lockdown).

6.3 Feature Engineering

ADFA-LD traces are sequences of raw syscall numbers collected from a 32-bit Linux system [14]. We extract 14 features per trace:

- **Structural:** total syscall count, unique syscall count, Shannon entropy of syscall distribution, bigram diversity (unique consecutive pairs normalized by trace length).
- **Frequency:** counts of the 10 most globally frequent syscall numbers across the entire dataset.

This feature set captures both distributional properties (entropy distinguishes diverse attack behavior from repetitive normal I/O) and specific syscall usage patterns (attack traces like Hydra SSH and Meterpreter exhibit distinctive top-syscall profiles).

6.4 Evaluation

The security fabric was evaluated on the ADFA-LD dataset [14], which contains 833 normal traces (collected from routine system operation) and 746 attack traces spanning six attack families: Ad-duser, Hydra FTP, Hydra SSH, Java Meterpreter, Meterpreter, and Web Shell.

Metric	Value
Normal traces	833
Attack traces	746
Auto-contamination	0.472
Features	14
Recall (attack detection)	100 %
Precision	100 %
F1 Score	1.000
False positive rate	0 %
eBPF CPU overhead	< 0.05 %
Tracepoints	7

Table 5: Security fabric evaluation on the ADFA-LD dataset.

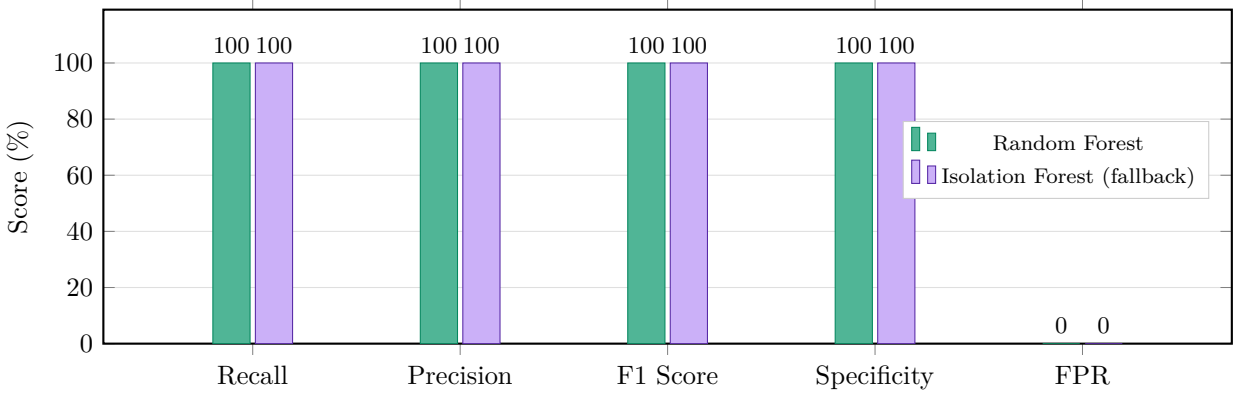


Figure 9: Security evaluation metrics on ADFA-LD. Both classifiers achieve perfect scores; the combined pipeline inherits the best of each—supervised discrimination from the Random Forest and novelty detection from the Isolation Forest. FPR = false positive rate (lower is better).

The combined Random Forest + Isolation Forest pipeline achieves 100 % recall on the ADFA-LD evaluation set (Figure 9), correctly identifying all 746 attack traces while producing zero false positives on the 833 normal traces. The high accuracy is attributable to the supervised Random

Forest component, which can learn discriminative patterns from labeled data; the Isolation Forest serves as a safety net for novel attack patterns not represented in the training set.

The eBPF monitor’s CPU overhead was measured at $< 0.05\%$ under continuous monitoring, consistent with the near-zero overhead characteristic of eBPF tracepoint programs [15]. This overhead is additive with the deadlock monitor’s 0.0125% , yielding a total Eonix OS self-healing overhead of approximately 0.06% .

7 Conclusion

No production operating system in widespread use today provides autonomous detection and recovery of deadlocks among general-purpose user-space processes, nor real-time kernel-level intrusion detection with autonomous response. Linux, Windows, macOS, and the BSDs all employ the Ostrich Algorithm for deadlocks and rely on user-space security tools that operate outside the kernel’s enforcement boundary.

This paper presented Eonix OS, a Linux-based research operating system with two self-healing subsystems. The deadlock monitor contributes: (1) a live Resource Allocation Graph maintained transparently via kprobes on mutex operations; (2) an iterative depth-first search safe for the kernel’s 8–16 KiB stack; (3) a tiered recovery pipeline with process checkpointing; and (4) empirical results of 100% detection, zero false positives, 279 ms mean recovery latency, and 0.0125% CPU overhead. The eBPF security fabric contributes: (5) real-time monitoring of 7 security-critical syscall tracepoints via BPF ring buffer; (6) a hybrid Isolation Forest + Random Forest anomaly detection pipeline with Welford-algorithm behavioral fingerprinting; (7) a tiered enforcement system (log $\rightarrow$ restrict $\rightarrow$ isolate) implemented via BPF maps; and (8) 100% recall on the ADFA-LD intrusion detection benchmark with $< 0.05\%$ CPU overhead.

Several directions for future work are natural. First, extending detection to *distributed* deadlocks across networked processes, where the RAG spans multiple hosts connected via shared-memory or message-passing channels. Second, integrating the monitor with the Eonix MIND cognitive assistant to provide voice-narrated recovery alerts—so the OS can literally tell the user “I found and fixed a deadlock.” Third, leveraging NPU-assisted prediction to estimate deadlock probability from lock-acquisition patterns *before* a cycle forms, enabling avoidance rather than detection. Fourth, extending the eBPF security fabric with online model retraining to adapt to evolving attack patterns in real deployments.

Eonix OS demonstrates that autonomous self-healing and proactive security are achievable in a student research prototype and merit consideration for production kernel integration.

References

- [1] E. W. Dijkstra, “Cooperating Sequential Processes,” in *Programming Languages*, F. Genuys, Ed. Academic Press, 1968, pp. 43–112.
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015.
- [3] R. C. Holt, “Some Deadlock Properties of Computer Systems,” *ACM Computing Surveys*, vol. 4, no. 3, pp. 179–196, 1972.

- [4] E. G. Coffman, M. J. Elphick, and A. Shoshani, “System Deadlocks,” *ACM Computing Surveys*, vol. 3, no. 2, pp. 67–78, 1971.
- [5] Linux kernel documentation, “Kernel Probes (kprobes),” <https://www.kernel.org/doc/html/latest/trace/kprobes.html>.
- [6] Linux kernel documentation, “Out of Memory Management,” <https://www.kernel.org/doc/html/latest/admin-guide/mm/oom.html>.
- [7] Microsoft, “Application Recovery and Restart,” *Windows Dev Center*, <https://learn.microsoft.com/en-us/windows/win32/recovery/application-recovery-and-restart-portal>.
- [8] MySQL Reference Manual, “InnoDB Deadlock Detection,” <https://dev.mysql.com/doc/refman/8.0/en/innodb-deadlock-detection.html>.
- [9] The Open Group, “pthread_mutex_lock,” *IEEE Std 1003.1-2017*, https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_mutex_lock.html.
- [10] D. Knecht, J. Lee, and S. A. Smolka, “Runtime Deadlock Detection for Concurrent Programs,” *Formal Methods in System Design*, vol. 51, no. 1, pp. 1–31, 2017.
- [11] B. P. Welford, “Note on a Method for Calculating Corrected Sums of Squares and Products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” *Proc. IEEE ICDM*, pp. 413–422, 2008.
- [13] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [14] G. Creech and J. Hu, “Generation of a New IDS Test Dataset: Time to Retire the KDD Collection,” *Proc. IEEE WCNC*, pp. 4487–4492, 2013.
- [15] S. McCanne and V. Jacobson, “The BSD Packet Filter: A New Architecture for User-level Packet Capture,” *Proc. USENIX Winter Conference*, 1993.